Memory Matching Game

Java Swing source code

```
import javax.swing.*;
import java.awt.*;
import java.util.ArrayList;
import java.util.Arrays;
import java.util.Collections;

public class MemoryMatchingGame extends JFrame {

    JButton[] cards = new JButton[16];

    String[] symbols = {
            "♦■", "♦■", "♠■", "♠■",
            "■", "■", "£", "£",
            "■", "■", "■■■", "■■■",
            "$", "$", "@", "@"
    };

    boolean[] matched = new boolean[16];

    int first = -1;
    int moves = 0;
    int pairs = 0;
    int seconds = 0;
    int attemptsLeft = 25;

    boolean busy = false;
    boolean gameOver = false;

    JLabel scoreLabel = new JLabel("", JLabel.CENTER);
    JLabel timeLabel = new JLabel("", JLabel.CENTER);
    JLabel attemptsLabel = new JLabel("", JLabel.CENTER);

    JPanel board;

    javax.swing.Timer timer;
    javax.swing.Timer delayTimer;

    public MemoryMatchingGame() {
        setTitle("Memory Matching Game");
        setSize(500, 550);
        setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
        setLocationRelativeTo(null);

        showMenu();

        setVisible(true);
    }

    // Main Menu
    void showMenu() {

        if (timer != null) {
            timer.stop();
        }

        if (delayTimer != null) {
            delayTimer.stop();
        }

        getContentPane().removeAll();
        setLayout(new BorderLayout());

        JPanel panel = new JPanel(new GridLayout(5, 1, 10, 10));

        panel.setBorder(
                BorderFactory.createEmptyBorder(40, 50, 40, 50)
        );

        JLabel title = new JLabel(
                "MEMORY MATCHING GAME",
                JLabel.CENTER
        );

        title.setFont(new Font("Arial", Font.BOLD, 23));

        JButton start = new JButton("Start Game");
        JButton instructions = new JButton("Instructions");
```

```
    JButton restart = new JButton("Restart Game");
    JButton exit = new JButton("Exit");

    panel.add(title);
    panel.add(start);
    panel.add(instructions);
    panel.add(restart);
    panel.add(exit);

    add(panel, BorderLayout.CENTER);

    start.addActionListener(e -> startGame());

    instructions.addActionListener(e ->
            JOptionPane.showMessageDialog(
                    this,
                    "HOW TO PLAY\\n\\n" +
                            "1. Click two cards to reveal symbols.\\n" +
                            "2. Matching pairs remain open.\\n" +
                            "3. Non-matching cards turn back over.\\n" +
                            "4. Match all 8 pairs to win.\\n" +
                            "5. You have 25 attempts to win.\\n" +
                            "6. Finish in less time and fewer moves!",
                    "Instructions",
                    JOptionPane.INFORMATION_MESSAGE
            )
    );

    restart.addActionListener(e -> startGame());

    exit.addActionListener(e -> System.exit(0));

    revalidate();
    repaint();
}

// Start or Restart Game
void startGame() {

    if (timer != null) {
        timer.stop();
    }

    if (delayTimer != null) {
        delayTimer.stop();
    }

    Arrays.fill(matched, false);

    first = -1;
    moves = 0;
    pairs = 0;
    seconds = 0;
    attemptsLeft = 25;

    busy = false;
    gameOver = false;

    getContentPane().removeAll();
    setLayout(new BorderLayout(10, 10));

    scoreLabel.setText("Moves: 0 | Pairs: 0/8");
    timeLabel.setText("Time: 0 seconds");
    attemptsLabel.setText("Attempts left: 25");

    ArrayList<String> shuffled =
            new ArrayList<>(Arrays.asList(symbols));

    Collections.shuffle(shuffled);

    JPanel top = new JPanel(new GridLayout(3, 1, 5, 5));

    top.add(scoreLabel);
    top.add(timeLabel);
    top.add(attemptsLabel);

    board = new JPanel(new GridLayout(4, 4, 6, 6));

    for (int i = 0; i < 16; i++) {

        final int index = i;
        final String symbol = shuffled.get(i);

        cards[i] = new JButton("?");
        cards[i].setFont(
```

```
            new Font("Arial", Font.BOLD, 28)
        );

        cards[i].setFocusPainted(false);

        cards[i].addActionListener(e -> {

            if (busy || gameOver || matched[index]
                    || cards[index].getText().equals(symbol)) {
                return;
            }

            cards[index].setText(symbol);

            // First card selection
            if (first == -1) {

                first = index;

            } else {

                // Second card selection
                moves++;
                attemptsLeft--;

                scoreLabel.setText(
                        "Moves: " + moves +
                                " | Pairs: " + pairs + "/8"
                );

                attemptsLabel.setText(
                        "Attempts left: " + attemptsLeft
                );

                int previous = first;
                first = -1;

                // Matching pair
                if (shuffled.get(previous).equals(symbol)) {

                    matched[previous] = true;
                    matched[index] = true;

                    pairs++;

                    scoreLabel.setText(
                            "Moves: " + moves +
                                    " | Pairs: " + pairs + "/8"
                    );

                    if (pairs == 8) {
                        endGame(true);
                    } else if (attemptsLeft == 0) {
                        endGame(false);
                    }

                } else {

                    // Non-matching pair
                    busy = true;

                    delayTimer = new javax.swing.Timer(
                            800,
                            event -> {

                                cards[previous].setText("?");
                                cards[index].setText("?");

                                busy = false;

                                if (attemptsLeft == 0 && !gameOver) {
                                    endGame(false);
                                }
                            }
                    );

                    delayTimer.setRepeats(false);
                    delayTimer.start();
                }
            }
        });

        board.add(cards[i]);
    }
    JButton restart = new JButton("Restart");
```

```
    JButton menu = new JButton("Main Menu");
    JButton exit = new JButton("Exit");

    restart.addActionListener(e -> startGame());
    menu.addActionListener(e -> showMenu());
    exit.addActionListener(e -> System.exit(0));

    JPanel bottom = new JPanel();

    bottom.add(restart);
    bottom.add(menu);
    bottom.add(exit);

    add(top, BorderLayout.NORTH);
    add(board, BorderLayout.CENTER);
    add(bottom, BorderLayout.SOUTH);

    timer = new javax.swing.Timer(1000, e -> {

        if (!gameOver) {
            seconds++;

            timeLabel.setText(
                    "Time: " + seconds + " seconds"
            );
        }
    });

    timer.start();

    revalidate();
    repaint();
}

// Win or Lose Condition
void endGame(boolean won) {

    if (gameOver) {
        return;
    }

    gameOver = true;
    busy = true;

    if (timer != null) {
        timer.stop();
    }

    if (delayTimer != null) {
        delayTimer.stop();
    }

    String message;

    if (won) {
        message = "CONGRATULATIONS! YOU WON!\\n\\n";
    } else {
        message = "GAME OVER! YOU LOST!\\n\\n";
    }

    message +=
            "Matching pairs: " + pairs + "/8\\n" +
                    "Total moves: " + moves + "\\n" +
                    "Attempts remaining: " + attemptsLeft + "\\n" +
                    "Time taken: " + seconds + " seconds";

    JOptionPane.showMessageDialog(
            this,
            message,
            "Game Finished",
            JOptionPane.INFORMATION_MESSAGE
    );

    int choice = JOptionPane.showConfirmDialog(
            this,
            "Would you like to play again?",
            "Game Finished",
            JOptionPane.YES_NO_OPTION
    );

    if (choice == JOptionPane.YES_OPTION) {
        startGame();
    } else {
        showMenu();
    }
```

```
    }

    public static void main(String[] args) {

        SwingUtilities.invokeLater(
                () -> new MemoryMatchingGame()
        );
    }
}
```